Multi-Scale Context Profiling for Position-Aware Retrieval-Augmented Generation

Ahmed HABBANI
Institut national des postes et télécommunications
Rabat, Morocco
ahmedhabbani207@gmail.com

Abstract

Large Language Models (LLMs) increasingly advertise long context windows, with recent systems extending their nominal context length to hundreds of thousands or even millions of tokens. However, nominal context length does not necessarily imply uniform context utilization. Prior work on the *Lost in the Middle* phenomenon has shown that models may retrieve information more reliably when it appears near the beginning or the end of the prompt, while performance can degrade when relevant evidence is placed in the middle.

In this paper, we propose a model-agnostic framework for **multi-scale context profiling** and **position-aware context orchestration**. Instead of assuming that all positions inside a context window are equally usable, the proposed method empirically estimates a model-specific reliability function over tested context lengths and evidence positions. The resulting Multi-Scale Profiling Grid is then used at inference time to route high-priority retrieved chunks toward empirically reliable prompt regions.

We evaluate the approach in a controlled key-value retrieval setting on Qwen2.5-0.5B-Instruct under hardware-constrained conditions using an NVIDIA RTX 4050 Laptop GPU. The profiling results reveal a clear U-shaped reliability curve, with high retrieval accuracy at the beginning and end of the prompt and substantially lower accuracy in the middle. A subsequent orchestration experiment shows that profile-guided placement improves partial evidence recovery compared with random and recency-biased placement, while adding no meaningful latency overhead. These results provide a proof-of-concept validation that context placement should be guided by empirical model-specific reliability rather than by fixed prompt templates.

Keywords— Retrieval-Augmented Generation, Long-Context Language Models, Lost in the Middle, Context Profiling, Prompt Orchestration, Position-Aware Retrieval.

I Introduction

Retrieval-Augmented Generation (RAG) has become a standard approach for grounding Large Language Models (LLMs) in external knowledge [1]. In industrial settings, RAG systems are commonly used to inject technical documentation, legal contracts, internal reports, or domain-specific knowledge into the prompt before generation. At the same time, recent LLMs have significantly expanded their nominal context windows, encouraging the assumption that larger prompts can be processed without major loss of fidelity.

However, long context capacity should not be confused with uniform context usability. Prior work on the *Lost in the Middle* phenomenon shows that the position of relevant evidence inside the prompt strongly affects retrieval performance [2]. In many cases, models retrieve information more accurately when it appears near the beginning or the end of the context, while performance drops when the same evidence is placed near the middle. This creates a critical weakness for RAG pipelines: even when the correct document chunk is retrieved, the model may fail to use it if it is injected into a low-reliability region of the prompt.

This issue becomes more important as context windows grow. A model may behave reliably at shorter prompt lengths, moderately degrade at intermediate lengths, and exhibit severe retrieval failures at longer lengths. Therefore, a single global estimate of context reliability is insufficient. The useful context topology of a model is not only position-dependent, but also scale-dependent.

To address this problem, we propose a **Multi-Scale Context Profiling** framework. The core idea is to evaluate the target model across multiple tested context sizes and multiple relative evidence positions. This produces a profiling grid that maps retrieval success as a function of both total prompt length and relative evidence depth. At runtime, a context orchestrator uses this grid to select the closest safe profile for the current request and rearranges the retrieved chunks accordingly.

Our contributions are threefold:

- We introduce a Multi-Scale Profiling Grid that captures retrieval reliability across both prompt length and relative evidence position.
- We propose a Nearest Upper Bound policy for selecting a conservative precomputed profile at inference time based on the raw token payload.
- We design a position-aware context orchestration strategy that routes high-priority chunks away from empirically weak regions and toward reliable prompt zones, without modifying or fine-tuning the underlying model.

II Related Work

A Long-Context Language Models

Recent advances in Large Language Models have significantly increased the nominal context window available at inference time. Several architectural and training-based techniques have been proposed to extend the maximum sequence length that a model can process. For instance, long-context methods such as LongRoPE aim to extend LLM context windows to extremely large token budgets [3]. However, increasing the maximum context length does not necessarily guarantee uniform context utilization. A model may technically accept a long input sequence while still failing to reliably retrieve or use information located in specific regions of the prompt.

This distinction between *nominal context length* and *effective context usability* is central to our work. Rather than proposing another method for extending the context window, we focus on measuring and exploiting the usable topology of an already deployed model.

B Lost in the Middle

The *Lost in the Middle* phenomenon describes the tendency of language models to use information more effectively when it appears near the beginning or the end of the context, while performance may degrade when the same information is placed in the middle [2]. This behavior has been observed in both multi-document question answering and key-value retrieval settings.

This finding is particularly important for Retrieval-Augmented Generation systems. In a standard RAG pipeline, the retriever may correctly select the relevant document chunk, but the generator can still fail to use it if the chunk is placed in a low-reliability region of the prompt. Therefore, retrieval quality alone is not sufficient: the placement of retrieved evidence inside the context window also matters.

Our work builds on this observation but extends it in two ways. First, we do not assume that the degradation pattern is constant across all prompt lengths. Second, we use the measured degradation profile as an actionable input for a runtime context orchestration module.

C Long-Context Evaluation Benchmarks

Several benchmarks have been introduced to evaluate long-context understanding. LongBench provides a bilingual and multi-task benchmark covering long-document question answering, multi-document question answering, summarization, few-shot learning, synthetic tasks, and code completion [4]. RULER further argues

that simple needle-in-a-haystack evaluation is insufficient for measuring true long-context understanding and introduces configurable synthetic tasks that vary both sequence length and task complexity [5].

Our framework is complementary to these benchmarks. Instead of proposing a new benchmark for comparing models globally, we propose a profiling mechanism for calibrating a specific model before deployment. The resulting profile is not only diagnostic; it is directly used by the context orchestrator to decide where retrieved chunks should be placed.

D Context Compression and Retrieval Optimization

Another line of work focuses on reducing the amount of context passed to the model. Retrieval, reranking, summarization, and context compression methods aim to keep only the most relevant information, thereby reducing token cost and improving response quality. These methods are especially useful when the retrieved corpus is large or when the model has a limited context window.

However, compression and retrieval optimization usually focus on *which* chunks should be included, rather than *where* they should be placed. Our approach is orthogonal to these methods. A RAG system can first retrieve and rerank relevant chunks, then use our Multi-Scale Profiling Grid to decide their placement inside the prompt. In this sense, our method adds a position-aware orchestration layer on top of existing retrieval pipelines.

E Position-Aware Context Orchestration

Prompt construction is often treated as a formatting problem: instructions, retrieved documents, and the user query are concatenated according to a fixed template. This can be suboptimal in long-context settings because the reliability of different prompt regions is not uniform.

We propose to treat prompt construction as a topology-aware routing problem. Given a query, a set of retrieved chunks, and a model-specific reliability profile, the orchestrator assigns high-priority chunks to empirically reliable regions of the context window. Unlike static prompt templates, this routing decision depends on the estimated total token length of the current request and on a precomputed multi-scale profile of the model.

III Problem Formulation

A Long-Context RAG Setting

We consider a Retrieval-Augmented Generation pipeline in which a user query Q is answered using a set of retrieved document chunks:

$$\mathcal{D} = \{d_1, d_2, \ldots, d_k\}. \tag{1}$$

Each chunk d_i is associated with a relevance score r_i, typically produced by a retriever or reranker. The final prompt given to the language model is constructed by concatenating system instructions, retrieved chunks, formatting tokens, and the user query.

Let the total prompt length be:

$$T = \mathrm{Len}(I) + \mathrm{Len}(Q) + \sum_{i=1}^{k} \mathrm{Len}(d_i), \tag{2}$$

where I denotes system instructions and prompt metadata, and $\mathrm{Len}(\cdot)$ represents the number of tokens.

In standard RAG pipelines, retrieved chunks are often inserted according to a fixed template or their retrieval order. This assumes that every position in the prompt is equally usable by the model. However, long-context models often exhibit position-dependent retrieval behavior. Therefore, two chunks with the same semantic relevance score may have different effective usefulness depending on where they are placed in the prompt.

B Relative Evidence Position

Let x_i denote the starting token index of a chunk d_i inside the final prompt. We define the relative position of d_i as:

$$p_i = \frac{x_i}{T}, \tag{3}$$

where $p_i \in [0, 1]$. A value of $p_i \approx 0$ means that the chunk is placed near the beginning of the prompt, while $p_i \approx 1$ means that it is placed near the end. The middle of the prompt corresponds to $p_i \approx 0.5$.

The central assumption of this work is that the probability of successfully using a chunk is not only a function of its semantic relevance, but also a function of its position and the total prompt length.

C Context Reliability Function

We define a context reliability function:

$$S(c, p) \in [0, 1], \tag{4}$$

where c is a tested context window size and p is a relative position inside the prompt. The value $S(c, p)$ represents the empirical probability that the model correctly retrieves or uses information placed at relative position p when the total context length is approximately c.

A high value of $S(c, p)$ indicates a reliable prompt region, while a low value indicates a position where the model is likely to ignore, misinterpret, or fail to retrieve the relevant information.

We define the unreliable region for a context size c as:

$$\mathcal{U}_c = \{p \in [0, 1] \mid S(c, p) < \tau\}, \tag{5}$$

where τ is a reliability threshold. Similarly, the reliable region is defined as:

$$\mathcal{R}_c = \{p \in [0, 1] \mid S(c, p) \geq \tau\}. \tag{6}$$

The threshold τ can be selected according to the desired robustness level of the application.

D Scale-Dependent Degradation

A key observation motivating this work is that the reliability function $S(c, p)$ is scale-dependent. The degradation curve observed at a short context length may not generalize to longer prompts. Therefore, instead of estimating a single position-reliability curve, we model reliability as a two-dimensional function over both total context length and relative evidence position:

$$\mathcal{G} = \{(c, p, S(c, p)) \mid c \in C, p \in P\}, \tag{7}$$

where C is the set of tested context windows and P is the set of evaluated relative positions. We call this structure the **Multi-Scale Profiling Grid**.

E Optimization Objective

Given a set of retrieved chunks $\mathcal{D}$ and their relevance scores $\{r_1, r_2, \ldots, r_k\}$, the objective of the context orchestrator is to construct a prompt layout that maximizes the expected usefulness of the injected evidence.

We define the expected utility of a chunk d_i placed at position p_i under context size c as:

$$U(d_i, p_i, c) = r_i \cdot S(c, p_i). \tag{8}$$

The global objective is then:

$$\max_{\pi} \sum_{i=1}^{k} r_i \cdot S(c, p_{\pi(i)}), \tag{9}$$

where π denotes a placement policy assigning each chunk to a position in the prompt.

F Practical Routing Constraint

In real-time systems, the orchestrator cannot run a full profiling evaluation for every user query. Therefore, the reliability grid is computed offline before deployment. At inference time, the orchestrator only estimates the raw token length of the request:

$$T_{\text{raw}} = \text{Len}(I) + \text{Len}(Q) + \sum_{i=1}^{k} \text{Len}(d_i). \tag{10}$$

It then selects the nearest tested context window that upper-bounds the current prompt length:

$$c^* = \min\{c \in C \mid c \geq T_{\text{raw}}\}. \tag{11}$$

The selected profile $\mathcal{M}_{c^*}$ is used to guide chunk placement. This conservative strategy avoids using an overly optimistic profile from a shorter context window.

IV Proposed Framework

We propose a two-stage framework for position-aware context construction in long-context Retrieval-Augmented Generation. The first stage is an offline profiling phase that empirically measures the reliability of a target model across multiple context lengths and evidence positions. The second stage is an online orchestration phase that uses the resulting profile to construct safer prompt layouts at inference time.

The framework is model-agnostic: it does not require access to internal attention weights, architectural modifications, or fine-tuning. It treats the language model as a black-box system and estimates its effective context topology through controlled probing.

A Framework Overview

The proposed architecture contains two main components:

- **ContextEvaluator**: an offline module responsible for building the Multi-Scale Profiling Grid. It evaluates the target model at several tested context windows and relative evidence depths.
- **ContextOrchestrator**: an online module responsible for constructing the final prompt. It estimates the raw token payload, selects the safest corresponding profile, and assigns retrieved chunks to reliable prompt regions.

First, the ContextEvaluator generates synthetic or semi-synthetic long-context prompts containing a known target fact at controlled positions. The model is queried, and its ability to recover the target fact is measured. This produces a reliability map over prompt length and evidence position. During deployment, the ContextOrchestrator uses this map to guide chunk placement for real user queries.

B Offline Multi-Scale Profiling

Let $C = \{c_1, c_2, \ldots, c_n\}$ be the set of tested context windows. These values are selected to cover the usable range of the target model. In principle, the tested windows can be selected geometrically:

$$C = \{8k, 16k, 32k, 64k, 128k\}, \tag{12}$$

or adapted to the actual maximum context length of the deployed model.

For each tested context window $c \in C$, the evaluator considers a set of relative evidence positions:

$$P = \{0.0, 0.1, 0.2, \ldots, 0.9, 1.0\}. \tag{13}$$

At each position $p \in P$, the evaluator constructs a prompt of approximate length c by inserting a target evidence statement inside distractor text. The target statement is placed so that its relative position is approximately p. The model is then asked a question whose answer depends exclusively on the inserted target statement.

The model response is scored using:

$$\mathrm{Eval}(y, a) = \begin{cases} 1, & \text{if the response } y \text{ correctly contains the answer } a, \\ 0, & \text{otherwise.} \end{cases} \tag{14}$$

For each pair (c, p), the experiment is repeated over N trials:

$$S(c, p) = \frac{1}{N} \sum_{j=1}^{N} \mathrm{Eval}(y_j, a_j). \tag{15}$$

The result of this phase is a Multi-Scale Profiling Grid:

$$\mathcal{G} = \{(c, p, S(c, p)) \mid c \in C, p \in P\}. \tag{16}$$

C Profile Storage and Runtime Selection

The profiling grid is stored as a lightweight configuration file. Each tested context length is associated with a reliability curve over relative positions. At inference time, the orchestrator estimates T_{raw}, then selects:

$$c^* = \min\{c \in C \mid c \geq T_{\mathrm{raw}}\}. \tag{17}$$

If T_{raw} exceeds the largest tested context window, the system can trigger compression, reduce the number of retrieved chunks, or reject the request depending on the application constraints.

D Reliable Zone Identification and Placement

Once the profile $\mathcal{M}_{c^*}$ is selected, positions are classified as reliable if:

$$S(c^*, p) \geq \tau, \tag{18}$$

and unreliable otherwise.

The retrieved chunks are sorted by relevance scores:

$$r_1 \geq r_2 \geq \cdots \geq r_k. \tag{19}$$

The orchestrator assigns high-priority chunks to positions that maximize:

$$U(d_i, p, c^*) = r_i \cdot S(c^*, p). \tag{20}$$

In practice, a greedy assignment strategy is sufficient: rank chunks by semantic relevance, rank available prompt slots by empirical reliability, and assign the most important chunks to the safest slots.

E Computational Complexity

The offline profiling phase can be computationally expensive because it requires multiple model calls across context lengths, positions, and trials. However, this cost is paid only once per model and deployment configuration.

At runtime, the method is lightweight. Profile selection requires a lookup over the tested context windows, with complexity $O(|C|)$ or $O(\log |C|)$ if the context lengths are sorted. Chunk placement requires sorting the retrieved chunks, resulting in $O(k \log k)$, where k is the number of retrieved chunks. Therefore, the orchestration layer introduces minimal overhead compared to long-context model inference.

V Experiments

This section evaluates the proposed position-aware context profiling and orchestration framework. The objective is twofold. First, we empirically estimate the reliability function $S(c, p)$ of a target language model under controlled evidence placement. Second, we test whether the resulting reliability profile can be used as an actionable signal for prompt construction.

The experiments are designed as a proof-of-concept rather than as a large-scale benchmark. They were conducted on a local machine equipped with an **NVIDIA RTX 4050 Laptop GPU**. This hardware constraint motivates the use of compact instruction-tuned models and moderate context lengths. In the final analysis, we focus on **Qwen2.5-0.5B-Instruct**, because it is the model that produced both an exploitable position-dependent reliability profile and a measurable improvement under profile-guided orchestration.

A Experimental Objectives

The experimental study addresses the following questions:

1. Does the tested model exhibit position-dependent retrieval reliability?
2. Can the measured reliability profile be used to identify weak and reliable regions of the prompt?
3. Does profile-guided placement improve evidence usage compared to random or recency-biased placement?
4. Does the orchestration layer introduce a measurable latency overhead?

The experiments are organized in two stages. The first stage is an offline profiling experiment based on a controlled key-value retrieval task. The second stage is an orchestration experiment comparing three prompt construction strategies: baseline-random placement, recency-biased placement, and profile-guided placement.

B Model and Runtime Setup

The main evaluated model is **Qwen2.5-0.5B-Instruct**. This model was selected because it is small enough to be executed on the available laptop GPU while still being able to follow retrieval-style instructions. Other compact models were initially tested during exploratory runs, but they were not retained for the final orchestration analysis because they either failed the controlled UUID retrieval task or did not provide a sufficiently exploitable profile.

All generations are performed with deterministic decoding:

$$\texttt{do\_sample=False}.$$

The evaluation is black-box: no internal attention weights, gradients, or model modifications are used. The model is accessed only through its generated output.

C Profiling Task: Single-Key Retrieval

The profiling stage uses a controlled key-value retrieval task. Each prompt contains a JSON object composed of n key-value pairs. Keys and values are randomly generated UUID strings. For each trial, one target key is selected and the model is asked to return the corresponding value.

The position of the target key-value pair is controlled across five relative locations:

$$p \in \{\text{beginning}, 25\%, \text{middle}, 75\%, \text{end}\}.$$

We evaluate:

$$n \in \{20, 50, 100\}$$

key-value pairs. For each pair (n, p), we run $N = 100$ independent trials.

D Orchestration Task: Two-Key Retrieval

After profiling, we evaluate whether the estimated reliability curve can guide prompt construction. To avoid reducing the profile-guided policy to a simple recency policy, the orchestration experiment uses a two-key retrieval task. Each prompt contains a JSON object with 20 key-value pairs and two target keys. The model must retrieve both corresponding values.

We compare three placement strategies:

- **Baseline-random:** the two target pairs are inserted at random positions.
- **Recency-biased:** the two target pairs are inserted near the end of the prompt, at approximately 75% and 100% of the context.
- **Profile-guided:** the two target pairs are inserted into the most reliable prompt regions identified by the profiling curve. For Qwen2.5-0.5B, these regions correspond to the beginning and the end of the prompt.

The primary metric is *both-values accuracy*:

$$\mathrm{BothAcc} = \frac{1}{N}\sum_{j=1}^{N} \mathbf{1}\left[a_j^{(A)} \subset y_j \wedge a_j^{(B)} \subset y_j\right]. \tag{21}$$

We also report *any-value accuracy*:

$$\mathrm{AnyAcc} = \frac{1}{N}\sum_{j=1}^{N} \mathbf{1}\left[a_j^{(A)} \subset y_j \vee a_j^{(B)} \subset y_j\right]. \tag{22}$$

Because the task is synthetic, we use a conservative unsupported-output rate:

$$\mathrm{UnsupportedRate} = 1 - \mathrm{AnyAcc}. \tag{23}$$

This metric counts outputs that contain neither of the expected values. It should not be interpreted as a full hallucination metric, but rather as a strict indicator that the generated response is unsupported by the target evidence.

VI Results

A Profiling Results

Table 1 reports the key-value retrieval accuracy of Qwen2.5-0.5B-Instruct across different context sizes and evidence positions.

Table 1: Single-key retrieval accuracy for Qwen2.5-0.5B-Instruct across context sizes and evidence positions.

Context size	Beginning	25%	Middle	75%	End
20 pairs	0.99	0.73	0.40	0.81	1.00
50 pairs	0.88	0.50	0.30	0.66	0.98
100 pairs	0.00	0.00	0.00	0.00	0.00

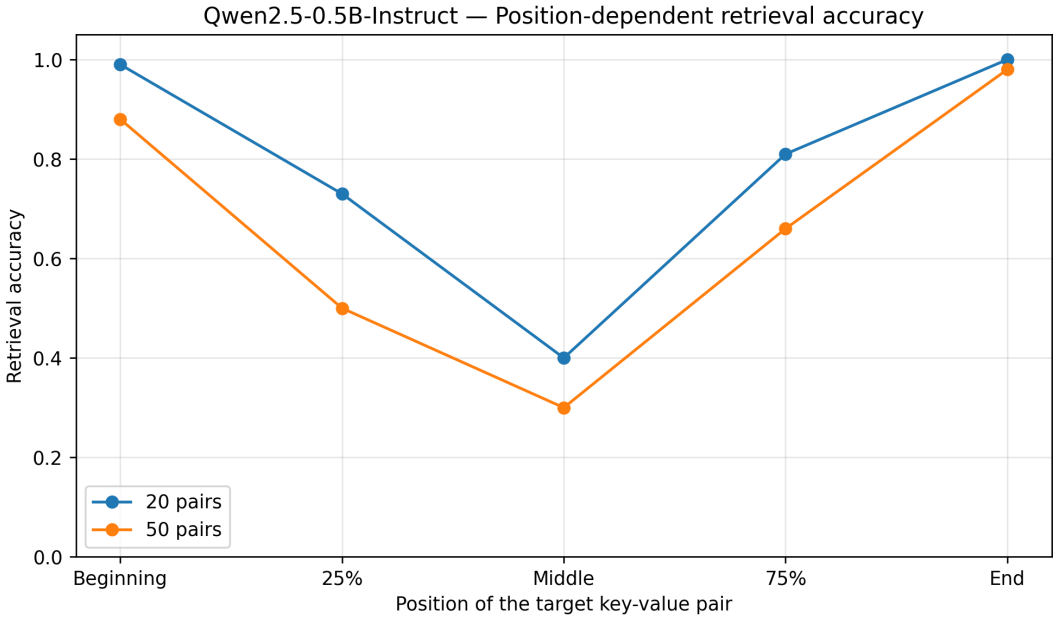

Figure 1: Position-dependent retrieval accuracy of Qwen2.5-0.5B-Instruct. The model retrieves evidence more reliably at the beginning and end of the prompt than in the middle.

The 20-pair setting reveals a clear U-shaped reliability pattern. The model achieves near-perfect retrieval when the target pair is placed at the beginning (0.99) or at the end (1.00) of the prompt, while accuracy drops to 0.40 when the same information is placed in the middle. The same tendency is visible at 50 pairs, where accuracy remains high at the beginning (0.88) and end (0.98), but decreases to 0.30 in the middle.

The 100-pair setting is not used to draw conclusions about positional reliability, because the input length reaches the truncation limit observed during the experiment. In this case, the model may not receive the full prompt, so the zero accuracies reflect a token-budget failure rather than a reliable measurement of context topology.

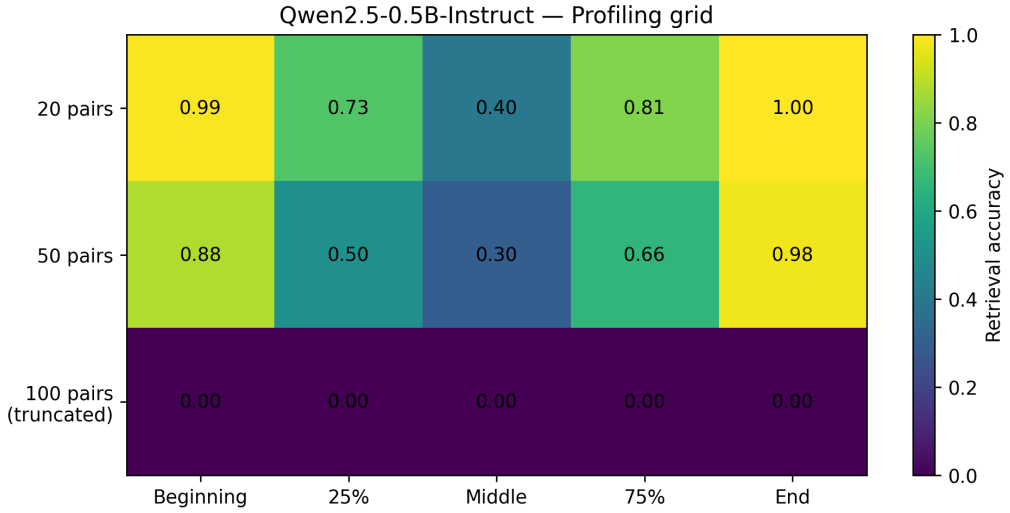

Figure 2: Multi-scale profiling grid for Qwen2.5-0.5B-Instruct. The 100-pair setting is shown for completeness but excluded from the main interpretation because it reaches the truncation limit.

B Position-Dependent Degradation

To quantify the degradation, we compare the middle-position accuracy with the average accuracy of the two extremities:

$$\Delta_{\text{middle}} = \frac{S(c, \text{beginning}) + S(c, \text{end})}{2} - S(c, \text{middle}).$$

For the 20-pair setting:

$$\Delta_{\text{middle}} = \frac{0.99 + 1.00}{2} - 0.40 = 0.595.$$

For the 50-pair setting:

$$\Delta_{\text{middle}} = \frac{0.88 + 0.98}{2} - 0.30 = 0.63.$$

This shows that the middle-position degradation is not marginal. The model loses approximately 60 percentage points of retrieval accuracy when the relevant evidence is moved from reliable extremity regions to the middle of the prompt.

C Orchestration Results

Table 2 reports the results of the two-key orchestration experiment for Qwen2.5-0.5B-Instruct. All strategies use the same number of key-value pairs and have comparable input lengths. No truncation is observed.

Table 2: Orchestration results for Qwen2.5-0.5B-Instruct on the two-key retrieval task.

Strategy	BothAcc	AnyAcc	Acc. A	Acc. B	Unsupported rate	Mean latency (s)	Mean input tokens
Baseline-random	0.11	0.42	0.30	0.23	0.58	2.55	1527.45
Recency-biased	0.12	0.38	0.20	0.30	0.62	2.49	1527.89
Profile-guided	0.14	0.65	0.17	0.62	0.35	2.50	1524.70

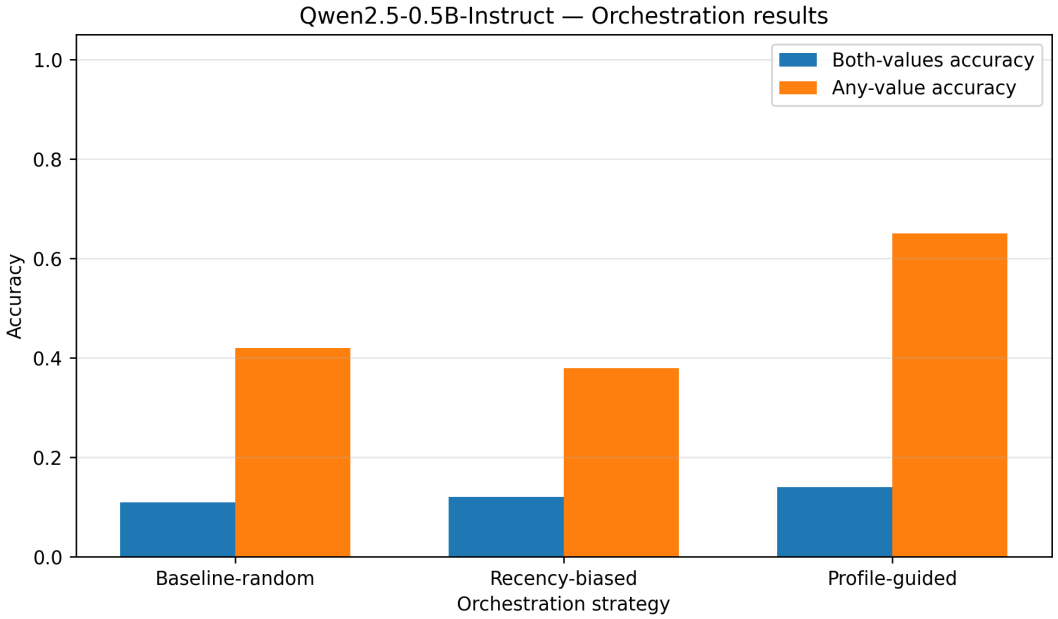

Figure 3: Comparison of baseline-random, recency-biased, and profile-guided orchestration. Profile-guided placement yields the strongest improvement in any-value accuracy.

The profile-guided strategy provides the best performance under both metrics. The improvement is modest for strict both-values accuracy, increasing from 0.11 under baseline-random placement and 0.12 under recency-

biased placement to 0.14 under profile-guided placement. This indicates that exact recovery of two UUID values remains difficult for the tested compact model.

However, the improvement is more substantial under any-value accuracy. The profile-guided strategy reaches 0.65, compared to 0.42 for baseline-random placement and 0.38 for recency-biased placement. In absolute terms, this corresponds to a gain of $+0.23$ over the random baseline and $+0.27$ over the recency-biased strategy.

D Recency Bias Is Not Sufficient

An important observation is that the recency-biased strategy does not outperform the random baseline under any-value accuracy. While placing information near the end of the prompt can be beneficial in single-evidence settings, the two-key task requires the model to recover multiple pieces of evidence. A purely recency-oriented layout concentrates the evidence near the end and does not exploit the reliable beginning region identified by the profiling phase.

By contrast, profile-guided placement uses both reliable extremities of the prompt. This supports the central claim of the paper: prompt construction should not rely on a fixed heuristic such as "put everything at the end", but should instead be guided by an empirical reliability profile of the target model.

E Latency Analysis

The latency results show that profile-guided orchestration does not introduce a meaningful runtime overhead. The mean generation latency is 2.55 seconds for baseline-random placement, 2.49 seconds for recency-biased placement, and 2.50 seconds for profile-guided placement.

Since all strategies use a similar number of tokens, the runtime cost is dominated by model inference rather than by the orchestration step. This is consistent with the proposed framework: the expensive profiling phase is performed offline, while runtime orchestration only requires a lookup in the precomputed reliability profile and a lightweight reordering of retrieved chunks.

F Summary of Findings

The experimental results support three main findings:

1. Qwen2.5-0.5B-Instruct exhibits a clear position-dependent reliability pattern, with higher retrieval accuracy at the beginning and end of the prompt and lower accuracy in the middle.
2. The measured reliability profile can be used as a practical signal for prompt construction. Profile-guided placement improves any-value accuracy in the two-key retrieval task.
3. The orchestration layer does not introduce a measurable latency overhead compared with random or recency-biased placement.

At the same time, strict both-values accuracy remains low, showing that exact multi-evidence UUID retrieval is still difficult for compact language models. The results should therefore be interpreted as a proof-of-concept validation of profile-guided orchestration rather than a complete solution to long-context reasoning.

VII Limitations

A Single Retained Model

The final experimental analysis focuses on Qwen2.5-0.5B-Instruct. This choice is motivated by the fact that it produced the clearest usable reliability profile and the only positive orchestration signal among the compact models tested during the exploratory phase. However, this also limits the generality of the empirical claims.

The proposed framework is model-agnostic, but the present experiments only demonstrate it on one retained model.

This limitation is partly due to hardware constraints. The experiments were run on an NVIDIA RTX 4050 Laptop GPU, which restricts the range of model sizes and context lengths that can be evaluated repeatedly. Each profiling run requires many model generations across positions and context sizes, and the orchestration stage adds additional controlled trials. Larger models and longer context windows would require substantially more memory and runtime.

B Synthetic Retrieval Task

The experiments use synthetic JSON key-value retrieval with UUID strings. This setting is useful because it isolates the effect of evidence position and avoids semantic shortcuts. However, UUID retrieval is also a strict exact-copying task. A model may identify the relevant region but still fail to reproduce the full UUID correctly. Therefore, the measured accuracy combines retrieval ability and exact copying ability.

Future work should complement this setup with shorter symbolic values, natural-language facts, and document-level question answering tasks.

C Limited Context Scale

The proposed framework is designed for multi-scale profiling across several context windows. In the present experiments, the usable context range is limited by hardware and truncation constraints. The 100-pair setting for Qwen2.5-0.5B reached the observed token limit and is therefore excluded from the main reliability analysis.

As a result, the current experiments validate the principle of position-aware profiling, but do not yet provide a full large-context grid over very long prompt lengths.

D Conservative Unsupported-Output Metric

The orchestration experiment reports an unsupported-output rate defined as $1 - \mathrm{AnyAcc}$. This metric is conservative and task-specific. It indicates that the generated answer contains neither of the expected target values, but it does not distinguish between hallucination, omission, formatting errors, or partial copying failures. A more complete hallucination analysis would require manual inspection or a task-specific unsupported-claim detector.

E No Real Corpus Retrieval

The current orchestration experiment controls the position of evidence directly inside a synthetic prompt. It does not yet include a full retrieval pipeline with embedding-based search, reranking, or real document chunks. Therefore, the results validate the placement component of the framework, but not the full end-to-end behavior of a production RAG system.

VIII Conclusion

This paper proposed a model-agnostic framework for position-aware context profiling and orchestration in Retrieval-Augmented Generation. Instead of treating the prompt as a uniform memory space, the proposed method estimates how reliably a target model retrieves information from different regions of its context window. The resulting Multi-Scale Profiling Grid is then used to guide the placement of high-priority evidence at inference time.

The experimental study on Qwen2.5-0.5B-Instruct provides a proof-of-concept validation of this idea. In a controlled key-value retrieval task, the model exhibits a clear U-shaped reliability curve: retrieval accuracy is high when the target evidence is placed at the beginning or end of the prompt, but drops substantially when the

same evidence is placed in the middle. This confirms that even compact instruction-tuned models may not use their context uniformly.

The orchestration experiment further shows that the measured profile can be used as a practical routing signal. Profile-guided placement improves partial evidence recovery compared with both random and recency-biased placement, while adding no meaningful latency overhead. Although strict two-value UUID retrieval remains challenging, the results support the central claim that context placement should be guided by empirical model-specific reliability rather than by fixed prompt templates.

Future work will extend this validation to larger models, longer context windows, real RAG corpora, and reasoning-oriented tasks. Another important direction is to combine position-aware orchestration with context compression, so that high-value evidence can be both selected and placed in the most reliable regions of the prompt.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[2] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[3] Y. Ding, L. Liu, X. Zhang, S. Huang, X. Xu, D. Xiong, and K. Chen, "Longrope: Extending llm context window beyond 2 million tokens," in *International Conference on Machine Learning*, 2024.

[4] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, "Longbench: A bilingual, multitask benchmark for long context understanding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[5] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg, "Ruler: What's the real context size of your long-context language models?" *arXiv preprint arXiv:2404.06654*, 2024.